极客时间《企业级 Agent 工程架构实战营》课程大纲

一、课程定位

这是一门面向已经具备大模型应用和基础 Agent 开发经验的工程师，帮助其从“能够实现一个 Agent Demo 或业务项目”，升级为“能够设计、评审、建设和持续演进企业级 Agent 系统”的架构进阶课程。

课程不以重复讲解 LLM API、Prompt Engineering、基础 Tool Calling、基础 RAG 或某个 Agent 框架的使用为主要目标，而是聚焦企业级 Agent 工程中真正决定系统可落地、可扩展、可维护的架构设计与工程治理问题，帮助学员完成从 Agent Developer 到 Agent Architect 的关键能力跃迁。

课程不仅关注如何开发企业级 Agent，更关注为什么这样设计、为什么这样组织团队、为什么这样与 AI 协作。它希望帮助学员建立 AI 时代真正具备长期竞争力的工程能力，而不是停留在某一种框架或技术栈的使用层面。

课程同时将架构选型与评测验证作为一条贯穿主线：每个阶段不仅讲“如何实现”，还要求学员比较候选方案、识别适用条件和关键取舍，并通过 Baseline、Badcase、Benchmark 与 ADR 为架构决策提供证据。课程将结合 DSH（DeepSeek Harness）等前沿开源 Harness 的源码设计，以及 Graph Engineering 对复杂执行拓扑的建模方法，帮助学员理解最新实践背后的稳定工程对象，而不是把课程绑定在某个框架或版本上。

课程将围绕企业级 Agent 系统设计中的核心架构问题展开：

- 如何建立对于 LLM、AI Agent 和 AI Native 的底层认知，理解 Agent 的本质、边界和未来演进方向，形成属于 AI 时代的软件工程思维；
- 如何理解 AI Coding 带来的研发范式变革，建立人与 AI 高效协作的方法论，掌握 AI Coding 的理念、技巧、Workflow 与 Infra，使 AI 成为工程能力的放大器，而不是简单的代码生成工具；
- 如何理解大厂 Agent 架构师真正需要具备的能力模型，建立从 Demo 开发到企业级系统设计的完整成长路径；

- 如何掌握 PE、CE、HE、LE、GE 等 Agent 调优技术体系，理解各种 Agent Engineering 技术背后的设计思想，而不是停留在经验式 Prompt 调整；
- 如何判断一个业务问题是否真正适合 Agent，并根据业务特点选择最恰当的 Agent 架构，发挥 Autonomous Agent 和 Workflow Agent 各自的优点；
- 如何完成企业级 Agent Runtime 的架构设计，将单机 Agent Loop 演进为支持长任务、状态持久化、故障恢复、水平扩展、多 Agent 协同和分布式执行的运行时系统；
- 如何建立企业级 Agent 的系统抽象能力，将 Model、Context、Knowledge、Tool、Skill、Memory 等能力从业务逻辑中解耦，形成可复用、可治理、可持续演进的基础能力体系；
- 如何建立企业级 Agent 的可观测、可回放、可评测和可持续演进体系，通过 Trace、Replay、Eval、Regression Gate 等工程机制持续提升系统质量，而不是依赖 Prompt 调试和经验排查。

课程最终训练的是企业级 Agent 的：

- 架构判断能力；
- 运行时设计能力；
- 系统抽象能力；
- 工程治理能力；
- 可靠性设计能力；
- 可观测与诊断能力；
- 生产演进能力；
- 架构交付与评审能力；

二、课程核心收获

1. AI Native 认知与 AI Coding 能力
    - 建立 LLM、AI Agent、Agent Engineering 的底层认知与统一技术框架；
    - 理解 AI Native 工程师、AI Native 团队和企业级 Agent 架构师的能力模型；
    - 建立 AI 时代的软件工程思维，理解人与 AI 的最佳协作范式；
    - 掌握 AI Coding 的理念、Workflow、Skill、Infra 与最佳实践，建立高效的人机协同研发模式；
    - 建立 Prompt、Context、Memory、Eval 等 Agent 调优技术的整体认知，理解其设计思想与适用边界；
    - 建立持续适应 AI 技术快速演进的学习框架与工程方法论。

2. 企业级 Agent 架构能力
    - 能够完成 Agent 系统分析，拆解目标、状态、决策、行动、观察、验证和停止条件；
    - **能够根据业务约束完成传统软件、LLM Feature、Workflow、Autonomous Agent、Graph 与混合架构之间的架构判断与技术选型；**
    - 能够建立 Runtime、Context、Memory、Knowledge、Tool 等核心对象的系统抽象能力；
    - 能够设计企业级 Agent Runtime，实现单 Agent 到多 Agent 的架构演进；
    - 能够设计控制权矩阵、自主性预算、人工介入边界和协作模式；
    - 能够输出系统架构图、状态机、时序图、ADR、Failure Matrix 等企业级架构设计成果。
3. Runtime 与平台工程能力
    - 能够设计统一的多模型 LLM Runtime 和企业级 Agent Runtime；
    - 能够将模型调用升级为可调度、可恢复、可回放、可扩展的运行时系统；
    - 能够设计 Run、Task、Step、Attempt、Action、Event、Checkpoint 等统一执行模型；
    - 能够建设 Scheduler、Queue、Worker、Sandbox、Session Store、Checkpoint Store 等核心基础设施；
    - 能够处理长任务、多租户、故障恢复、重复执行、副作用控制和水平扩展等企业级问题。
4. 企业级工程治理能力
    - 能够建立 Model、Context、Knowledge、Tool、Skill、Memory 等核心能力的统一治理体系；
    - 能够建立 Context Engineering、Memory Engineering、Tool Engineering 等 Agent Engineering 最佳实践；
    - 能够治理权限、安全、凭据、副作用、资源隔离等企业级工程问题；
    - 能够建立统一的规范、接口、生命周期和演进策略，支撑多个 Agent、多个业务和多个团队持续协作；
    - 能够构建可复用、可维护、可持续演进的企业级 Agent 基础设施。
5. 评测与受控进化能力
    - 能够建立 Outcome、Trajectory、System 三层 Eval 与质量保障体系；
    - 能够构造 Dataset、Fixture、Trial、Grader 等企业级评测基础设施；
    - 能够利用 Trace、Replay、Failure Taxonomy 等手段完成复杂 Agent 系统的问题诊断与根因分析；
    - 能够建立自动化 Benchmark、Regression、Offline Eval 等持续评测能力；
    - 能够通过 Canary、Rollback、CI Regression Gate 等工程机制实现 Agent 系统安全、稳定、受控演进。

6. AI Agent 架构师核心能力
    - 建立企业级 Agent 系统的架构判断能力，而不仅仅是技术实现能力；
    - 建立复杂系统抽象能力，能够从业务需求中提炼 Runtime、状态流、事件流和生命周期；
    - 建立架构设计、技术取舍、方案表达和技术评审能力；
    - 建立面向长期演进的软件工程思维，而不是一次性的 Demo 开发思维；
    - 具备独立负责企业级 Agent 系统设计、建设与持续演进的综合能力。

三、贯穿项目设计

课程提供一个统一的 B 端贯穿项目。所有学员围绕同一业务场景持续迭代，从模块化单体逐步演进为支持多租户、长任务、云端 Sandbox、微服务部署和持续评测的企业级 Agent 系统。项目统一承载课程中的 LLM Runtime、分布式 Agent Runtime、Context、Tool、Memory 与 EvalOps 能力。

贯穿项目：企业级多租户零售经营分析与决策 DataAgent

该项目面向品牌商家和零售企业，为多个企业租户提供经营数据分析、异常诊断、决策建议和受控业务执行能力。项目不止实现自然语言查数，而是让 Agent 能够围绕真实经营问题，自主完成分析规划、数据查询、Python 分析、证据验证和行动闭环。

典型任务

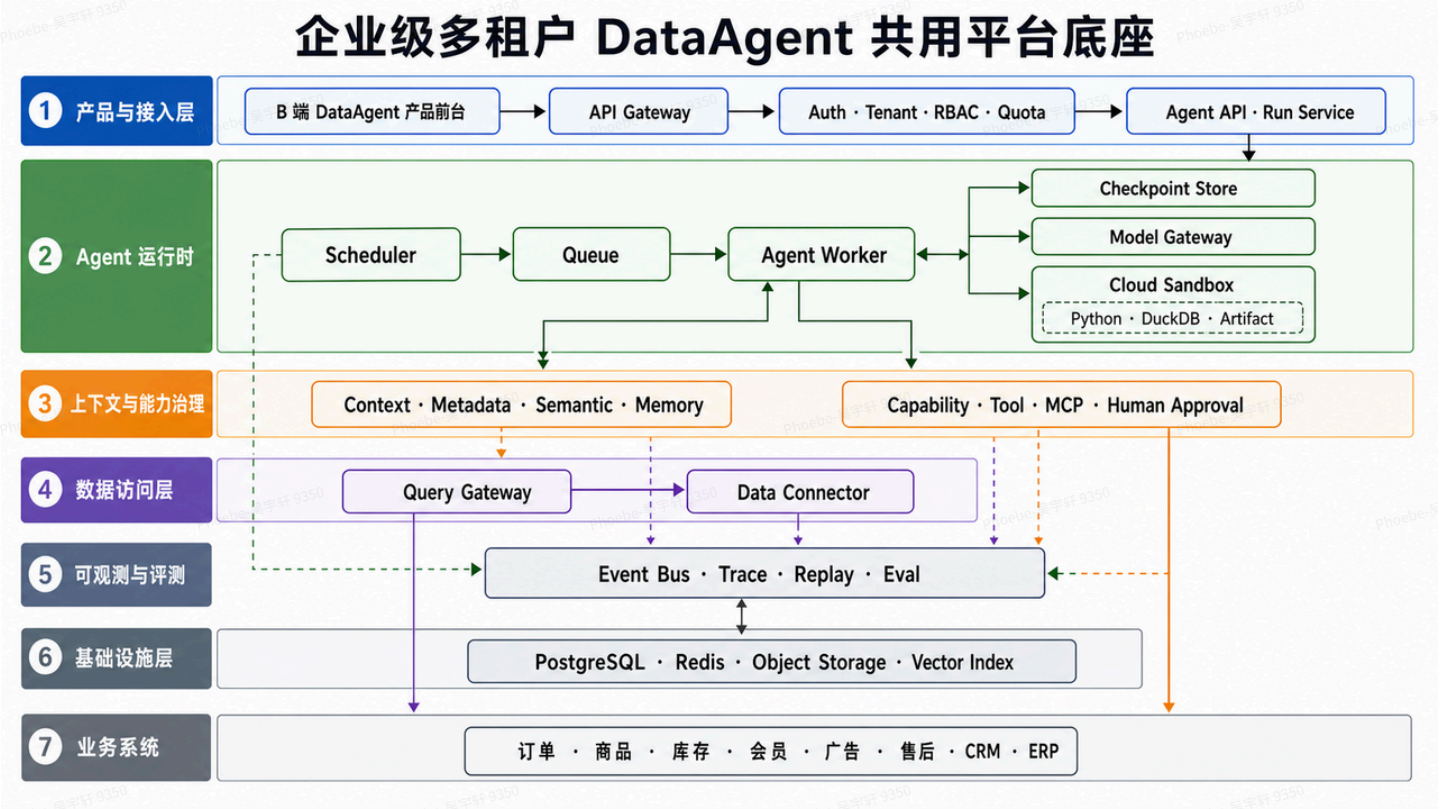
企业用户提交一个跨数据源与业务系统的经营问题，例如：

- 查询订单、商品、库存、会员、广告和售后数据；
- 理解企业指标口径，自主生成多步骤分析计划；
- 通过 SQL 与云端 Python Sandbox 完成数据查询、统计分析和图表生成；
- 诊断 GMV、毛利、转化率、退款率或库存等指标异常，并形成证据链；
- 生成补货、营销、人群运营或预算调整等行动建议；
- 对创建工单、调整预算和写入业务系统等操作发起人工审批；
- 周期性生成经营报告、监测异常，并持续跟踪行动效果；
- 由管理员配置租户级数据源、指标、模型、工具、预算和权限。

需要解决的架构问题

- 多租户、组织角色与数据权限；
- 数据源凭据和查询结果隔离；
- 指标、维度、业务术语和数据血缘治理；
- 行列级权限、只读 Query Gateway 与 SQL 安全；
- Workflow 与 Autonomous Analysis Loop 混合编排；
- SQL 执行与 Python Sandbox 隔离；
- 数据新鲜度、质量验证与证据追踪；
- 有副作用工具的风险分级与 Human Approval；
- 经营结论、执行轨迹、审计日志与回归门禁。

共用平台底座



企业级多租户 DataAgent 共用平台底座架构图

课程按照演进式架构推进：

1. 先用模块化单体建立业务、数据与效果基线；
2. 拆出统一的 LLM Runtime；
3. 拆出 Agent API、Scheduler、Worker 和 Sandbox；

4. 逐步增加 Data Connector、Query Gateway、Context、Semantic、Tool 和 Memory 等治理模块；
5. 最后补齐 Trace、Eval、多租户策略、安全隔离和发布治理。

四、详细课程大纲

第一篇：AI 核心认知篇：从工程到 AI Agent 必须建立的底层认知

---

1. 从训推机理理解 LLM：建立 AI Agent 开发的底层心智模型

- 核心概念深解（Agent 设计师必须内化的基本元素）：
  - Token/Tokenizer/Prompt/Reasoning 等
- 从训练数据 Schema 理解模型训练过程及能力边界与行为偏好：
  - PreTrain：通过与训练模型习得了哪些能力
  - SFT：指令跟随、工具调用/结构化输出、ReAct、Planning、Skill 的训练与代价
  - RLHF：对齐原理，收益与代价
- 从推理机制深入了解 Agent 底层的行为约束：
  - 一次完整的推理流程：建立对模型内部运作的直观认知
  - 理解自回归生成：prefill、decode、kvcache 机制与推理效率和成本的影响
  - 采样机制：Temperature/Top-p/Top-k/Logprobs 的机制与应用
- 工程开发必须建立的底层认知
  - 关于幻觉和不确定性：理解、接纳、应用幻觉和不确定性
  - 理解 ICL、窗口长度和注意力机制并灵活运用
  - Prompt 和安全的基本认知
  - 模型与确定性系统的职责边界：从效果、成本、时延和风险判断哪些能力依靠模型，哪些必须通过 Contract、Tool、Workflow 和验证机制补齐
  - 必须了解的 LLM 发展的行业认知

2. 从 Agent 本质到核心范式：Autonomous Agent 与 Workflow Agent 的优劣对比

- Agent 定义与核心概念的共识与非共识
- Agent 的两种根本范式机制详解：Autonomous 和 Workflow
- Autonomous Agent 多种范式详解、对比

    - ReAct：通过“决策—行动—观察”形成局部执行循环；
    - Plan-and-Execute：分离全局任务规划与局部步骤执行；
- Workflow Agent 多种范式详解、对比
    - Prompt Chaning
    - Routing
    - Parallelization
    - Orchestrator-Workers
    - Evaluator-optimizer
    - Agentic Workflow
- 企业级 Agent 架构的融合演进趋势、优劣对比、适用场景与选型考量

3. 拆解 AI Agent 工程师核心能力模型：找到最优晋级之路

- 从市场招聘需求分析洞察企业需要的人才画像与趋势
- 企业级 Agent 工程师的三维能力画像与模型
    - **技术能力**：软件工程、LLM 与 Agent 机制、Context、Tool、Memory、Runtime、Eval 与生产治理；
    - **架构能力**：任务建模、控制权分配、状态设计、可靠性、安全、成本与性能取舍；
    - **职业能力**：创新实验、结果责任、执行闭环、跨职能协作和持续学习
- 从初级到资深、架构师的能力分级
    - **L1 功能开发**：在既有架构下完成 Agent 功能与基础验证；
    - **L2 业务交付**：独立负责一条包含 Context、Tool、State 和 Eval 的业务链路；
    - **L3 系统设计**：统筹 Runtime、可靠性、安全、成本与可观测性；
    - **L4 平台演进**：建设共享基础设施、工程标准和持续评测体系。
- 架构师的核心工作方式：从“给出一个方案”升级为“提出候选方案、明确约束、验证取舍”，并使用工具完成架构交付与评审
- 从传统工程到资深 AI Agent 工程师的能力补全地图
- AI 时代 Agent 人才能力需求的变与不变

4. AI Native 团队与个人的知与行：

- AI Native 的定义、本质与伪 Native 认知与鉴别
  - AI-Enabled 与 AI Native 的本质区别
  - 案例对照：真实案例，看见 AI Native 的画面
- 建立 AI Native 的思维
  - 思维转换：完成任务→对结果负责；从执行者→定义者；从构建者→审查者；
  - 案例对照：真实案例，感受思维转换后的直观变化
- 个人 AI Native：成为一个人的 OPC
  - 角色、目标的转换
  - 建立个人 AI Agent 团队与资产体系：
  - 建立个人 Evaluation 体系和持续演进机制
  - 案例对照：超级个体的画面感
- 团队 AI Native：从个人效率到组织能力提升
  - 人、Agent 与确定性系统的责任边界与协作架构选型：不同风险等级下工具、执行者、审查者和责任人的控制权分配
  - AI Native 团队的基础设施：团队级规则、SPEC、Skill 与验证体系
  - 个人资产与团队资产的隔离边界、管理模式探讨
  - 案例对照：超级团队的画面感

5. 从根本讲透 LLM 调优技术：PE、CE、HE、LE、GE

- 五张图建立 PE/CE/HE/LE/GE 和彼此关系的初印象
- 从 OpenAI 五级 AI 发展路线图看 LLM 调优技术的产生背景和归宿
  - 五层技术的关系：递进、嵌套还是取代?
  - 模型内化趋势：模型能够内化什么，不能内化什么?
  - 长期学习重点：该学什么，不该学什么，掌握稳定的工程对象和判断方法
  - 架构选型：什么时候使用 Prompt、Context、Harness、Loop 或 Graph，什么时候保持简单
- 两种视角看待和使用五种技术的异同：
  - User 视角：用 PE 表达目标、CE 准备信息、HE 调用工具、LE 迭代验证、GE 编排复杂任务
  - Builder 视角：将五层能力固化为产品接口、Runtime、权限策略、反馈机制和评测体系
  - 双重视角：既能驾驭 AI 完成任务，也能把个人方法建设成可靠的 Agent 系统

第二篇：底层能力篇：掌握企业级 AI Coding 的道法术器

1. 企业级 AI Coding 之道：建立以人为本的生产观、角色观和责任观

- 个人级 AI Coding 和企业级 AI Coding 的根本区别
  - 多维度横向对比：从目标→方法→结果→反馈迭代等
  - 企业级 AI Coding 有效产出公式
- 企业级 AI Coding 之道的详细拆解和案例阐述：
  - 企业级 AI Coding 的四层架构：道法术器的本质、核心问题、关系
  - 案例：典型案例、标志性画面
- 企业级 AI Coding 三观：
  - 角色观：
    - 从“写代码的人”升级为“软件交付系统的设计者”
    - 从“亲自 Coding”转向“定义、编排与审查”
  - 生产观：
    - 从“Prompt 决定质量”转向“工程系统决定质量”
    - 从“节省 Token”转向“优化总交付成本”
    - 从“一次任务完成”转向“每次任务都形成复利资产”
  - 责任观：
    - 从“追求全自动”转向“风险适配的自动化”

2. 企业级 AI Coding 之法：把概率性生成转化为确定性交付

- 从 Prompt 驱动到方法驱动：
  - 为什么复杂工程任务不能依赖“一条好 Prompt”？
  - AI Coding 的四类不确定性：目标、上下文、执行与验证
  - 企业级方法论：SPEC、Context、Contract、Plan、Test、Loop、Verify、Deliver、Distill
  - 方法、技法、工具和平台能力的边界、适用场景
- SDD：从模糊需求到可执行规格
- CE：构造最小充分上下文
- Contract-First 与 Invariant-Driven Development
- Task Graph：复杂任务的拆解与编排
- Test-Driven 与 Eval-Driven Development

- Loop Engineering：建立受控的执行闭环
- Independent Verification：独立验证与证据驱动审查
- Progressive Delivery：从代码完成到安全上线

3. 企业级 AI Coding 之术：掌握高成功率的 Agent 驾驭与编排技巧

- 从方法论到实战技巧
  - 企业级 AI Coding 之术的八个控制变量
  - 如何同时优化效果、效率、成本和人类注意力
- 高质量 Prompt 与任务控制技巧与案例讲解
- 高信号密度的上下文工程技巧与案例讲解
- 任务拆解与执行节奏控制技巧与案例讲解
- Loop Engineering 高级技巧与案例讲解
- Hook、Skill 与确定性自动化组合技巧与案例讲解
- SubAgent 与多 Agent 的编排技巧与案例讲解
- 验证、恢复与成本优化

4. 企业级 AI Coding 之器：构建可治理、可验证、可复用的 AI Coding Harness

- 从 Coding Agent 到 AI Coding Harness
  - 单点工具、工具链与企业级生产系统的区别
  - “器”的本质：把经验、流程和制度外置为基础设施
- Context System：让 Agent 获得最小必要的信息
  - Agent-Friendly Repository、权威信息源、规则继承、冲突与过期治理
  - Context Manifest 与任务可复现性
- Sandbox 与执行环境：Wortree、多 Agent 并行、秘钥与环境变量治理、从开发者本机运行到企业级执行集群
- Workflow、Hook 、Skill 与 Agent 编排基础设施
- Observability、成本与组织治理：Agent Trace、工具调用、AI 领导力、从 Token 监控升级为软件生产效能观测
- Skill、规则与评测资产平台
  - Prompt、Rule、Skill、Workflow 和 Eval 的资产边界

    - 版本、所有者、适用范围和生命周期
    - 个人资产向团队资产的升级机制
    - Badcase 回流和持续评测
    - 建立随模型共同进化的组织 AI 资产飞轮
- 综合实战：搭建最小可用 AI Coding Harness
    - 为真实项目建立 Agent-Friendly Repository
    - 配置任务、上下文、工具权限和执行环境
    - 组合 Builder、Reviewer、Hook 和 Stop Gate
    - 从个人 Coding 效率升级为团队生产能力

第三篇：贯穿实战立项篇：把真实业务问题转化为可验证、可演进的 企业级 Agent 项目

1. 实战项目立项：企业级多租户零售经营分析与决策 DataAgent

- 项目 Demo 展示：实际跑起来的项目多个系统界面、操作演示，展示企业级项目的组成
- 项目业务与目标详解
    - 查询订单、商品、库存、会员、广告和售后数据；
    - 理解企业指标口径，自主生成多步骤分析计划；
    - 通过 SQL 与云端 Python Sandbox 完成数据查询、统计分析和图表生成；
    - 诊断 GMV、毛利、转化率、退款率或库存等指标异常，并形成证据链；
    - 生成补货、营销、人群运营或预算调整等行动建议；
    - 对创建工单、调整预算和写入业务系统等操作发起人工审批；
    - 周期性生成经营报告、监测异常，并持续跟踪行动效果；
    - 由管理员配置租户级数据源、指标、模型、工具、预算和权限。
- **项目系统架构详解**
    - 为什么从模块化单体起步，以及 Autonomous Analysis Loop、Workflow、Execution Graph 与确定性系统的职责边界
    - 模型执行、任务运行、上下文、能力、记忆和评测等核心模块的演进路径
- **项目架构选型机制：**每个里程碑提出候选方案，通过 Decision Matrix、Benchmark、Badcase 与 ADR 记录和验证关键取舍
- **项目质量契约与 M0 评测基线**

    - 从 Outcome、Trajectory、System 三个层次定义初始成功标准与首批关键 Eval Case
    - 记录 M0 版本的任务质量、证据完整性、成本、时延、安全与失败类型
- 项目脚手架和准备资源
- 贯穿课程机制介绍

第四篇：LLM 执行底座篇：构建面向 Agent Harness 的可靠模型执行 LLM Runtime

> 🍞 目标：让学员完成企业级 LLM Runtime 的设计与实现，建立 Agent Harness 稳定、可靠、可治理的模型执行底座。

1. 从散落的模型调用到 LLM Runtime：建立稳定而不失真的模型执行边界

> **本课目标：** 将 DataAgent 中所有模型调用统一收敛到 LLM Runtime，建立 Agent Harness 可以长期演进的企业级模型执行底座。

- **为什么不能在业务代码里直接调用模型 SDK：** 理解企业级 LLM Runtime 的定位，以及与 SDK Wrapper、Model Gateway、Agent Runtime 的职责边界。
- **如何让所有模型调用拥有统一契约：** 建立统一的模型调用接口、数据结构和 Provider Adapter，让上层 Agent 不再感知具体模型。
- **如何既统一接口，又保留不同模型的能力差异：** 设计兼容 Structured Output、Reasoning、Tool Calling、Streaming 等能力的模型抽象。
- **企业级 LLM Runtime 应该如何分层：** 建立 Registry、Routing、Execution、Observability 等模块，形成可持续演进的执行底座。
- **DataAgent 实战：对比直接 SDK、共享 Wrapper 与统一 Runtime 三种方案，将分散调用统一改造为企业级模型执行入口，并记录 M0 模型调用基线与 ADR。**

---

2. 多模型路由：用能力契约和策略路由选择正确模型

> **本课目标：** 为 DataAgent 建立可解释、可治理的多模型路由体系，让模型选择从"写死模型"升级为"按任务智能决策"。

- **为什么企业不能把模型名称写死在代码里：** 建立逻辑模型与物理模型分层，解除业务与具体模型的耦合。
- **如何根据任务自动选择最合适的模型：** 建立 Capability、Task Profile 与模型能力契约，实现按任务智能路由。
- **如何平衡效果、成本与稳定性：** 构建企业级模型路由、Fallback 与降级策略，而不是简单切换模型。
- **如何满足不同租户的模型策略要求：** 建立多租户模型权限、预算与区域治理能力。
- **DataAgent 实战：** 为分析、SQL、证据判断、报告生成等不同任务建立候选路由策略，并通过效果、成本、时延和稳定性对比完成选型。

---

3. 建立高并发下可靠的模型执行机制

> **本课目标：** 为 DataAgent 构建可靠模型执行机制，确保模型调用具备可恢复、可验证、可追踪的企业级可靠性。

- **为什么模型调用成功并不代表任务成功：** 建立 Logical Call、Attempt 等企业级执行模型，正确理解一次模型调用的生命周期。
- **模型调用失败后应该如何恢复：** 建立 Retry、Fallback、Timeout、Deadline 等可靠执行机制，而不是简单重试。
- **如何保证模型输出一定符合预期：** 建立 Structured Output、Schema Validation 与 Output Contract，阻止无效结果进入下游。
- **如何正确处理流式输出：** 建立流式执行与最终提交机制，避免中间结果影响业务逻辑。
- **DataAgent 实战：** 为模型调用注入真实故障与高并发测试，完成 Reliable Executor 的企业级实现。

---

4. 从能调用到可运营：建立企业级模型治理体系

> **本课目标：** 为 DataAgent 建立完整的模型治理能力，实现模型调用可追踪、可运营、可审计、可持续演进。

- **如何让每一次模型调用都可追踪、可定位问题：** 建立企业级 Trace、调用链和可观测体系。

- **如何真正管住模型成本：** 建立预算控制、成本统计、多租户成本归集等运营能力。
- **如何保证模型升级不会悄悄影响线上结果：** 建立模型版本、路由策略版本与 Replay 机制，实现可回溯治理。
- **如何支撑企业级多租户运营：** 建立租户隔离、权限控制、模型治理与安全审计体系。
- **DataAgent 实战：** 完成 LLM Runtime 的生产级验收，实现模型调用全链路可追踪、可治理、可运营。

第五篇：Agent Harness 与 Agent Runtime 工程架构篇：设计自主 + Workflow 的企业级混合系统

📌 基于第四篇已经完成的 LLM Runtime，将 DataAgent 中的一条核心业务链路：**从单进程、同步执行的 Agent Demo，逐步演进为支持以下能力的企业级执行系统**：

- Autonomous Analysis Loop;
- Durable Workflow;
- 统一 Run 状态与事件模型；
- 异步长任务；
- Checkpoint 与 Resume；
- Queue 与 Worker；
- Human Approval;
- 副作用幂等；
- 多租户资源治理；
- Run Timeline 与故障解释。

1. *Agent Runtime 的混合架构选型与设计：取长补短*

> **本课目标：** 将一条经营分析需求拆解成模型、代码与人工协同完成的企业级执行链路。

- **Task Contract：让业务需求变成可执行任务**——解决 Agent 不知道什么算完成的问题。
- **执行范式选型：单 Loop、固定 Workflow、动态 Graph、多 Agent Graph 与混合架构如何选择？**——解决所有任务套用同一种编排范式的问题。

- **控制权矩阵：哪些交给模型，哪些必须交给代码和人？**——解决 AI 边界不清的问题。
- **Graph Engineering：**如何设计节点、边、状态、Verifier、Human Gate 和恢复路径？——解决复杂执行拓扑不可见、不可评审的问题。
- **Autonomy Budget：如何限制 Agent 的自主性？**——解决无限循环、无限花钱、无限调用的问题。
- **DataAgent 实战：完成经营分析主链设计**——建立整个项目后续所有 Runtime 的执行骨架。

---

2. *从 while 循环到 Autonomous Harness：构建可验证的经营分析闭环*

> **本课目标：** 让 DataAgent 能够自主分析，同时始终受到验证、预算和停止条件约束。

- **Harness 为什么不是 Agent Loop？**——解决 Demo 能跑、生产不能跑的问题。
- **Plan、Replan 与 Verifier：如何保证分析过程可信？**——解决模型不断试错却越跑越偏的问题。
- **Stop Condition：什么时候应该停止？**——解决 Agent 不知道何时结束的问题。
- **Analysis State：如何管理分析过程？**——解决开放分析过程无法恢复、无法解释的问题。
- **DataAgent 实战：实现 Autonomous Analysis Loop**——完成经营分析的自主推理闭环。

---

3. *从聊天历史到统一执行模型：让每一步可持久化、可追踪、可重试*

> **本课目标：** 建立统一 Runtime 执行模型，让 Agent 的状态脱离聊天记录，实现可恢复、可追踪的运行时。

- **Run、Step、Attempt：统一 Runtime 语言**——解决执行状态无法表达的问题。
- **Event 与 Checkpoint：如何保存运行过程？**——解决崩溃后只能重跑的问题。
- **Run State Machine：统一生命周期管理**——解决状态混乱、无法恢复的问题。
- **Run Timeline：如何解释 Agent 每一步在做什么？**——解决企业运维和审计不可观测的问题。
- **DataAgent 实战：建立统一 Runtime Domain**——让所有 Agent、Workflow、SubAgent 使用统一执行模型。

---

4. *用 Durable Workflow 收口自主分析：实现中断、审批与副作用闭环*

> **本课目标：** 将 Autonomous Harness 纳入 Durable Workflow，实现审批、中断恢复与安全执行。

- **Workflow Shell：为什么开放分析必须放进 Workflow？**——解决 Agent 无法安全执行真实业务的问题。
- **Checkpoint 与 Resume：如何实现可恢复执行？**——解决长任务中断后无法继续的问题。
- **Human Approval：如何让人参与关键决策？**——解决高风险操作无法治理的问题。
- **副作用控制：如何保证业务操作不会重复执行？**——解决重复调用导致真实业务出错的问题。
- **DataAgent 实战：完成预算审批执行流程**——实现企业级经营决策闭环。

---

5. *从单进程到分布式 Runtime：让长任务可调度、可接管、可隔离*

> **本课目标：** 将 DataAgent 演进为支持异步调度、水平扩展和故障接管的分布式 Runtime。

- **Agent Runtime 分层：为什么不能同步执行？**——解决 Agent 无法支撑长任务的问题。
- **Scheduler、Queue、Worker：如何调度任务？**——解决企业级任务调度的问题。
- **Lease 与 Heartbeat：如何避免多个 Worker 重复执行？**——解决分布式并发一致性的问题。
- **多租户资源治理：如何保证公平运行？**——解决企业 SaaS 环境下资源隔离的问题。
- **DataAgent 实战：完成 Runtime 分布式部署**——实现可调度、可接管的 Agent Runtime。

---

6. *以故障为教材：证明 DataAgent 能跨故障完成任务*

> **本课目标：** 通过主动注入故障，验证整个 DataAgent Runtime 的可靠性，而不是证明它能正常运行。

- **Failure Taxonomy：企业级 Agent 都会遇到哪些故障？**——解决没有统一故障模型的问题。

- **Retry、Resume、Replay、Fork：恢复机制有什么区别？**——解决"重新运行"概念混乱的问题。
- **Runtime Invariant：系统有哪些绝不能破坏的原则？**——解决恢复过程中数据一致性的问题。
- **Run Timeline：如何完整解释一次故障？**——解决生产问题无法定位的问题。
- **DataAgent 实战：完成全链路故障注入测试**——证明 Runtime 能恢复、能接管、能解释。

第六篇：Agent 上下文治理篇：为每一步决策构建最小充分、可信可控的上下文

1. *从 Prompt 拼接到 Context Runtime：先定义每一步决策的信息契约*

> 本课目标：将 DataAgent 的“大 Prompt 拼接”重构为面向不同决策步骤、可治理且可验证的 Context Runtime。

- **上下文不等于 Prompt：**解决聊天历史、任务状态、业务知识和工具结果混在一起，导致信息不可治理的问题。
- **面向决策组织上下文：**解决 DataAgent 的规划、查询、分析和建议生成共用一套上下文模板的问题。
- **定义上下文输入契约：**明确每个步骤必须看到、可以看到和禁止看到哪些信息。
- **建立统一 Context Schema：**让每条信息携带来源、版本、权限、可信度和生命周期。
- **拆分 Context Provider：**将租户身份、任务目标、运行状态和业务知识从 Prompt 构造逻辑中解耦。
- **关键上下文缺失时停止执行：**防止 DataAgent 在指标口径、租户身份或权限不明确时自行猜测。
- **DataAgent 实战：**为分析规划、查询设计、证据验证和建议生成建立四套 Decision Context Contract。

---

2. *经营语义上下文：让 DataAgent 先理解指标，再接触数据*

> 本课目标：建立 DataAgent 的企业语义层，让 Agent 在查询数据前先准确理解指标口径、分析维度和权限边界。

- **数据库字段不等于业务语义：**解决模型根据表名和字段名猜测 GMV、退款率等指标含义的问题。

- **建立指标统一定义：**解决不同租户、部门和报表使用不同指标口径的问题。
- **管理业务术语和别名：**解决“销售额”“成交额”“GMV”等自然语言表达无法准确映射的问题。
- **明确分析粒度与时间语义：**解决日、周、自然月、时区和币种混用导致的错误分析。
- **建立 Data Catalog 与血缘：**让每个经营结论能够追溯到具体数据源、字段和计算规则。
- **权限先于相关性：**防止 DataAgent 检索或使用当前租户、角色无权访问的表和字段。
- **管理数据新鲜度与质量：**避免使用延迟、缺失或异常数据解释实时经营问题。
- **DataAgent 实战：**建设 GMV、订单数、客单价、转化率、退款率等核心指标的 Metric Registry。

---

3. *动态上下文装配与预算：让不同 Step 只看到最小充分信息*

> 本课目标：实现 DataAgent 的动态 Context Assembler，在有限 Token 和成本预算内保留当前决策最需要的信息。

- **上下文装配管线化：**解决 Context 构造依赖一次性字符串拼接、无法定位错误的问题。
- **硬性过滤与软性排序：**确保权限、租户和有效期不能被“高相关性”绕过。
- **必需信息优先保障：**防止工具描述、历史记录等内容挤掉指标口径和当前任务状态。
- **分区管理 Token Budget：**分别为规则、任务、状态、业务语义、证据和输出预留空间。
- **上下文去重与冲突处理：**解决同一指标多个版本、重复证据和相互矛盾信息同时进入模型的问题。
- **可信压缩与 Artifact 引用：**避免大型查询结果、表格和日志反复占用上下文窗口。
- **模型调用前充分性检查：**判断信息是否足够支撑当前决策，而不是“能调用就调用”。
- **DataAgent 实战：**分别为规划、查询、原因分析和建议生成设计动态 Context Profile。

---

4. *企业知识与 Agentic Evidence：围绕信息缺口构建可核验的证据包*

> 本课目标：让 DataAgent 围绕经营分析中的信息缺口持续查询，并将结果组织成可以支撑具体结论的 Evidence Package。

- **企业知识可用性治理：**解决文档已经入库，但因为解析、版本、权限和元数据问题仍然不可用的情况。

- **按内容结构进行知识解析：**避免表格丢失表头、指标文档丢失公式、查询结果丢失参数。
- **面向信息缺口选择数据源：**让 Agent 根据问题选择指标库、数据库、经营规则或历史 Artifact。
- **从单次检索升级为 Agentic Search：**解决一次 Top-K 检索无法覆盖复杂经营问题的问题。
- **建立搜索预算和停止条件：**防止 Agent 为补充证据无限查询、重复搜索和成本失控。
- **区分假设、证据与结论：**防止未经验证的推测被直接写入最终经营报告。
- **建立 Claim-Evidence 关系：**让每条经营结论都能说明由哪些数据和规则支撑。
- **DataAgent 实战：**围绕 GMV 下滑构建“流量—转化—客单价—库存—退款”的假设树和证据链。

---

5. *长任务上下文治理：让数小时运行和多 Agent 协作不丢目标、不重做、不漂移*

> 本课目标：让 DataAgent 在长时间运行、中断恢复和多 Agent 协作中持续保持正确的目标、状态、约束和证据。

- **区分 History、State、Checkpoint 和 Summary：**解决所有过程信息都依赖聊天历史保存的问题。
- **管理上下文生命周期：**防止只在当前 Step 有效的信息被错误带入整个 Run 或后续 Session。
- **按用途生成不同摘要：**解决步骤摘要、阶段摘要、交接摘要和用户报告相互混用的问题。
- **定义压缩保留不变量：**确保目标、未完成任务、关键约束、失败记录和证据引用不会被摘要删除。
- **大型对象进入 Artifact Store：**避免 SQL 结果、图表和日志在每轮调用中重复传输。
- **协作拓扑选型：**比较共享完整上下文、显式 Handoff、Artifact 引用与 Parent-Child Run，把目标、权限、证据和状态作为沿执行图边传递的受控载荷。
- **恢复时重新装配上下文：**避免中断后简单重放聊天历史，导致重复查询或使用过期权限。
- **显式设计 Parent-Child Handoff：**防止子 Agent 获得完整父任务上下文和不必要的数据权限。
- **检测 Context Drift：**发现目标偏移、事实失真、任务重复和证据来源丢失。
- **DataAgent 实战：**完成月度经营分析任务的中断恢复、阶段压缩和库存分析 Child Run 交接。

---

6. *Context Security 与 ContextOps：让上下文策略可审计、可回放、可回归*

> 本课目标：为 DataAgent 建立上下文安全、追踪和评测体系，使每次信息选择都可解释、可复现、可持续改进。

- **建立上下文信任边界：**解决系统规则、用户输入、企业知识和外部内容被当成同一信任等级的问题。
- **分离指令与外部数据：**防止企业文档、网页和工具结果中的恶意文本控制 Agent 行为。
- **防止跨租户上下文泄漏：**确保候选池、缓存、日志、Trace 和评测数据都遵守租户隔离。
- **建立 Context Decision Trace：**记录候选信息、过滤原因、排序、预算、压缩和最终装配结果。
- **建立上下文故障分类：**区分没有找到、权限过滤、排序丢失、预算挤压、压缩失真和模型推理错误。
- **建立 Context Eval 指标：**评估关键上下文召回率、噪声率、新鲜度、引用正确性和 Token 效率。
- **使用 Replay 比较策略：**在固定候选信息下比较不同 Policy、Ranker 和 Assembler 的效果。
- **上下文组件独立版本化：**避免一次 Context Policy 修改造成 DataAgent 行为不可预期地整体变化。
- **DataAgent 实战：**注入恶意文档、过期指标、跨租户数据和预算挤压等 Badcase，完成安全与回归测试。

第七篇：Agent 工具 / 技能治理篇：让 Agent 安全、可靠、高效地连接真实世界

> 本篇目标：为 DataAgent 建立从能力接入、工具调用到真实业务执行的治理体系，让 Agent 不仅能够调用工具，还能安全、可靠、可审计地完成真实行动。

---

1. *从 Function Call 到 Capability Architecture：一项能力应该以什么形态存在？*

> 本课目标：为 DataAgent 建立统一的能力地图，正确选择 API、Tool、Skill、Workflow、SubAgent 和 MCP，避免能力抽象混乱。

- **能力形态选型：**解决一个业务能力究竟应该封装成 Tool、Skill、Workflow 还是 SubAgent 的问题。

- **插件化与服务化选型：**对照 DSH 的 Everything is a Plugin，理解进程内模块、插件、远程服务和 MCP 的边界与代价。
- **模型决策与系统执行分离：**解决内部函数、模型工具和真实业务接口职责混杂的问题。
- **工具粒度与边界设计：**解决工具过粗不可控、工具过细难选择的问题。
- **最小模型可见面：**解决大量工具同时暴露导致误选、越权和 Context 浪费的问题。
- **DataAgent 能力地图实战：**完成数据查询、Python 分析、报告生成、工单创建和预算调整等能力的架构选型。

---

2. *Tool Contract 与动态工具供给：让 Agent 选对、传对并读懂结果*

> 本课目标：为 DataAgent 建立标准化 Tool Contract，让模型能够正确选择工具、生成参数并理解执行结果。

- **三层 Tool Contract：**解决模型理解、Runtime 执行和企业治理要求混在一个工具定义中的问题。
- **工具语义与参数设计：**解决工具名称模糊、参数自由度过高导致模型误调用的问题。
- **标准结果与错误语义：**解决 Agent 无法判断应该重试、修改参数还是停止的问题。
- **动态最小工具集：**解决不同租户、角色和任务阶段看到相同工具造成的越权与误选问题。
- **DataAgent 工具链实战：**将指标查询、经营数据查询、Python 分析和报告生成封装为企业级只读工具。
- **Tool Badcase 对照：**比较万能工具、底层 SQL 工具和业务语义工具，理解不同设计对调用成功率和安全性的影响。

---

3. *从 Tool Call 到 Action Runtime：把高风险行动收进受控执行闭环*

> 本课目标：为 DataAgent 建立“提议—校验—授权—审批—执行—验真”的 Action Runtime，防止模型直接操作真实业务系统。

- **Tool Call 与真实执行解耦：**解决模型生成参数后立即产生业务副作用的问题。
- **Action 生命周期：**解决校验、审批、执行、验证和补偿机制彼此割裂的问题。

- **风险分级与控制权设计：**解决查询、创建工单和调整预算采用相同安全策略的问题。
- **最小权限与短期凭据：**解决 Agent 长期持有生产系统账号和密钥的问题。
- **幂等与状态一致性：**解决重复执行、状态过期和部分成功导致的业务错误。
- **DataAgent 预算调整实战：**实现预算调整预览、人工审批、受控执行和结果验证闭环。
- **Action Badcase 演示：**通过重复请求、参数篡改、接口假成功和部分执行验证企业级安全边界。

---

4. *Skill Engineering：把一次成功的经营分析固化为领域能力*

> 本课目标：将 DataAgent 的经营异常诊断过程沉淀为可复用、可验证、可版本化的生产级 Skill。

- **Skill 的任务边界：**解决把 Prompt、Tool 或整个 Agent 都错误包装成 Skill 的问题。
- **Skill 六层结构：**解决 Skill 只有操作步骤，却缺少使命边界、领域原则和完成标准的问题。
- **Tool、Workflow 与 SubAgent 组合：**解决复杂 Skill 内部调用关系不可见、权限失控和循环依赖的问题。
- **渐进式加载：**解决 Skill 数量增加后全部进入 Context，导致 Token 成本和干扰持续增长的问题。
- **Skill Validator：**解决模型声称任务完成，但经营结论缺少指标、证据或验证的问题。
- **Skill 生命周期治理：**解决 Skill 修改后无法评估兼容性、效果退化和迁移影响的问题。
- **DataAgent Skill 实战：**将“指标确认—提出假设—查询证据—分析验证—原因排序—生成报告”固化为经营异常诊断 Skill。

---

5. *MCP 与 CapabilityOps：让跨团队能力可接入、可运营、可审计*

> 本课目标：将 DataAgent 的本地工具和远程 MCP 能力纳入统一治理，实现跨系统能力的安全接入与全生命周期运营。

- **MCP 的正确定位：**解决把 MCP 误认为业务治理、安全和权限完整解决方案的问题。
- **MCP 适用边界：**解决单体内部简单函数也被强行协议化、增加架构成本的问题。
- **生产级 MCP Server：**解决 MCP Server 只有 Tool Schema，却缺少认证、租户隔离、限流和审计的问题。

- **Capability Registry：**解决系统不知道有哪些能力、谁负责、哪个版本可以用于生产的问题。
- **Capability Seam 对照：**比较 DSH 插件接口与 MCP 远程协议在进程内扩展、跨进程接入和治理责任上的差异。
- **动态 Tool Set 编译：**解决 MCP Server 暴露大量工具后全部交给模型选择的问题。
- **Credential Broker 与 Tool Trace：**解决生产凭据泄漏，以及工具调用无法追责和审计的问题。
- **DataAgent MCP 实战：**将广告平台查询与预算调整能力接入独立 MCP Server，并由 Host 控制模型可见工具和真实执行权限。
- **全链路红队测试：**通过跨租户访问、伪造结果、重复执行和版本漂移，验证整个工具治理体系是否真正生效。

第八篇：Agent Memory 治理篇：让 Agent 有选择地记住、正确地回忆

> 本篇目标：为 DataAgent 建立可信、可控、可审计的长期记忆系统，让 Agent 能够跨任务复用有效信息，同时避免记错、用错、过期和越权。

---

1. *从聊天记录到记忆契约：确定 DataAgent 该记什么、不该记什么*

> 本课目标：明确 Agent Memory 的系统边界，建立可执行的记忆分类与写入策略。

- **Memory 与 Chat History、Context、State 的边界：**解决不同信息都被当成长记忆保存，导致系统职责混乱的问题。
- **权威数据与长期记忆的边界：**解决 DataAgent 将指标口径、实时经营数据和业务状态错误复制到 Memory 的问题。
- **记忆价值与风险判断：**解决模型仅凭“以后可能有用”就自动保存信息的问题。
- **Memory Policy：**解决什么信息允许写入、是否需要确认、保存多久以及谁可以使用的问题。
- **DataAgent 记忆对象设计：**从经营分析任务中区分用户偏好、历史任务引用、流程经验候选和禁止写入内容。
- **企业级记忆不变量：**确保 Memory 不替代权威数据源、不继承历史权限、不覆盖当前任务要求。

---

2. *从 Run 事件到可信记忆：构建可审计、可幂等的写入链路*

> 本课目标：将 DataAgent 任务中产生的候选信息，经过验证、去重和冲突处理后安全写入长期记忆。

- **记忆写入触发机制：**解决 Agent 在推理中途或每轮对话后随意修改长期记忆的问题。
- **Memory Write Pipeline：**通过提取、标准化、校验、策略判断和提交，解决记错、记脏和缺少证据的问题。
- **Memory Schema 与 Scope：**解决一条记忆属于哪个用户、租户、项目和任务类型的问题。
- **写入幂等与重复治理：**解决 Runtime 重试、消息重复消费导致同一记忆被多次写入的问题。
- **冲突、版本与失效：**解决用户偏好变化、新旧事实冲突以及旧记忆错误覆盖新记忆的问题。
- **DataAgent 写入实战：**从经营诊断 Run 中写入报告偏好和任务引用，拒绝写入临时结论、原始查询结果和模型推测。
- **企业级存储分层：**使用关系存储管理权限和版本，向量索引只负责召回，Evidence Store 保存原始证据。

---

3. *从向量 Top-K 到 Recall Contract：让记忆在正确 Step 进入 Context*

> 本课目标：建立面向任务步骤的受控召回机制，让 DataAgent 只在正确场景使用正确记忆。

- **Recall Contract：**解决所有任务步骤都使用同一套向量检索逻辑的问题。
- **租户、用户与权限过滤：**解决跨用户、跨项目和跨租户记忆泄漏的问题。
- **时效与状态过滤：**解决过期、删除、失效和冲突记忆继续参与任务的问题。
- **企业级召回排序：**综合相关性、范围、可信度、新鲜度和来源质量，解决只依赖向量相似度的问题。
- **召回、注入与行动权限分离：**解决历史记忆直接影响 SQL、预算调整和真实业务执行的问题。
- **Context Budget 与最小注入：**解决记忆过多挤占上下文、增加成本并干扰模型判断的问题。
- **DataAgent 召回实战：**在规划阶段引用历史行动，在报告阶段使用表达偏好，但重新查询当前经营数据和权限。
- **Memory 使用追踪：**记录召回候选、过滤原因和最终注入结果，解决记忆使用过程无法解释的问题。

4. *从个性化功能到 MemoryOps：证明记忆有用、无害、可纠正、可删除*

> 本课目标：建立 Memory 的控制面、安全防线和评测体系，使长期记忆成为可运营的企业级能力。

- **记忆查看与解释：**解决用户不知道 Agent 记住了什么、为什么记住以及在哪里使用的问题。
- **记忆修正与版本治理：**解决错误记忆无法纠正、修改后历史关系不可追溯的问题。
- **删除与失效传播：**解决主库删除后，向量索引、缓存和下游系统仍能召回旧记忆的问题。
- **Memory Poisoning 防护：**解决恶意文档、工具结果和模型误判污染长期记忆的问题。
- **多租户 MemoryOps：**解决租户管理员、普通用户和 Agent 之间的记忆管理权限边界问题。
- **Memory Eval：**同时评估写入准确率、召回价值、过期率、污染率、删除有效性和跨租户泄漏。
- **DataAgent 红队实战：**验证恶意指标口径注入、跨租户召回、历史授权误用和删除残留等 Badcase。
- **企业级上线验收：**通过 Memory Off、Blind Top-K 和 Governed Memory 对照实验，证明记忆确实提升任务效果且没有引入不可接受的风险。

第九篇：Agent 评测与受控进化篇：用科学的评测方法论驱动持续有效进化

> 本篇目标：为 DataAgent 建立从质量定义、自动评测、失败诊断到发布门禁的 EvalOps 闭环，让每一次系统优化都有证据、可复现、可回归、可安全上线。

---

1. *从“看起来不错”到质量契约：定义 DataAgent 什么才算真正成功*

> 本课目标：为 DataAgent 建立统一的质量标准，明确一个经营分析任务怎样才算做对、做稳并且可以进入生产。

- **从业务目标到 Eval Contract：** 解决团队对“任务成功”理解不一致、评测标准无法落地的问题。

- **Outcome、Trajectory、System 三层评测：** 解决只看最终答案、忽略执行过程与系统代价的问题。
- **关键行为与安全不变量：** 解决答案正确但存在越权查询、错误数据或危险操作的问题。
- **软评分与硬门槛：** 解决总分提升掩盖跨租户泄漏、未审批执行等严重风险的问题。
- **DataAgent 质量契约实战：** 为“华东区 GMV 下降诊断”定义结果质量、证据链、执行路径、成本与安全标准。

---

2. *从样例集合到可复现基准：构建不会欺骗自己的 Dataset 与 Fixture*

> 本课目标：为 DataAgent 建立贴近真实业务且环境固定的评测基准，让不同版本之间的效果比较真实、稳定、可复现。

- **从真实业务构建 Task Taxonomy：** 解决评测集只覆盖正常任务、无法反映生产长尾问题的问题。
- **企业级 Eval Case 设计：** 解决开放式 Agent 任务没有唯一标准答案、难以准确验收的问题。
- **Fixture 与环境快照：** 解决数据、时间、权限和工具变化导致评测结果不可复现的问题。
- **版本元组与结果归因：** 解决模型、Prompt、Context、Tool 等同时变化后无法判断提升来源的问题。
- **评测集分层治理：** 解决日常调优污染测试集、历史能力被新版本破坏的问题。
- **DataAgent 基准集实战：** 构建经营诊断、数据缺失、工具故障、跨租户访问和审批拒绝等核心 Case。

---

3. *从“有标准”到“能自动测”：建设可信的 Multi-Grader 与 Batch Eval Runner*

> 本课目标：将 DataAgent 的质量标准转化为可以批量运行、自动评分和持续回归的企业级评测系统。

- **根据判断对象选择 Grader：** 解决所有任务都依赖 LLM Judge、评分不稳定且缺乏事实验证的问题。
- **Code、Environment、Trace 与 LLM Grader：** 解决“Agent 说做了”和“真实环境确实完成了”无法区分的问题。

- **Grader 校准与版本治理：** 解决裁判模型自身漂移、误判和评分标准变化的问题。
- **Multi-Trial 与稳定性评测：** 解决单次运行偶然成功、无法代表真实系统能力的问题。
- **多维 Scorecard：** 解决单一总分掩盖质量、安全、成本和时延之间取舍的问题。
- **DataAgent 自动评测实战：** 建设 Batch Eval Runner，自动验证指标计算、证据有效性、执行轨迹和环境状态。

---

4. *从“分数下降”到“根因被证明”：用 Trace、Replay 与 Ablation 完成故障诊断*

> 本课目标：让学员能够把 DataAgent 的评测失败还原为具体机制问题，并通过受控实验证明真正根因。

- **评测结果与统一 Trace 关联：** 解决只能看到分数下降、无法定位错误发生在哪一步的问题。
- **企业级 Failure Taxonomy：** 解决所有失败都归因于模型或 Prompt、修错系统层次的问题。
- **症状、直接原因与根因：** 解决看到哪里错就修改哪里、不断积累局部补丁的问题。
- **Replay、Fork 与 Ablation：** 解决根因判断依赖经验猜测、缺乏对照实验证据的问题。
- **Regression Case 沉淀：** 解决线上问题只修复一次、后续版本再次出现的问题。
- **DataAgent 根因实验实战：** 定位错误指标口径来自模型、Context、Tool 还是跨租户 Memory 污染，并验证修复效果。

---

5. *从离线提升到安全发布：建立 Regression Gate 与受控进化闭环*

> 本课目标：为 DataAgent 建立从候选版本评测到灰度发布、异常回滚和线上反馈回流的受控进化机制。

- **Agent 资产统一版本治理：** 解决只管理代码版本、忽略模型、Prompt、Skill、Memory 和策略变更的问题。
- **Baseline 与 Candidate 公平比较：** 解决评测环境不同却把结果变化错误归因于候选版本的问题。
- **CI Regression Gate：** 解决平均分提升就直接发布、关键能力和安全能力悄悄退化的问题。
- **Shadow 与 Canary 发布：** 解决离线 Fixture 无法完整模拟真实业务分布和外部依赖的问题。

- **Kill Switch 与完整版本回滚：** 解决出现问题时只能回滚代码、无法恢复完整 Agent 行为的问题。
- **AI 自动优化的责任边界：** 解决 Agent 自动修改权限、安全规则或发布策略带来的失控风险。
- **DataAgent 发布实战：** 完成候选版本评测、门禁阻断、修复验证、租户灰度和异常回滚全流程。

第十篇：完结篇：面向未来的 Agent 工程判断

> **本篇目标：** 通过 DataAgent 最终展示完成全课程技术收口，重新理解 Agent 时代人的角色与责任，并建立面向未来一年持续判断技术变化的能力。

1. DataAgent 最终展示与全课程总结：从 Agent Demo 到企业级 Agent 系统

> **本课目标：** 通过完整业务演示、故障演示和架构复盘，证明 DataAgent 已经从一个能运行的 Demo，演进为可以承担真实业务的企业级 Agent 系统。

- DataAgent 最终业务闭环展示
- 正常、故障、演进三条链路演示
- 从 M0 到最终版本的架构演进复盘
- 一张全景图串起全课
- 三条企业级治理主线总结
- 从 Agent Developer 到资深 Agent Developer 总结

2. 人与 AI 的四种关系：当 AI 从工具变成执行者，人的责任发生了什么变化？

> 本课目标：帮助学员识别自己与 AI 的真实协作关系，并理解 AI 承担更多执行后，人应该保留哪些判断与责任。

- **人与 AI 的四种关系图谱：一张坐标图厘清人与 AI 关系的终极命题**
- **工具关系——AI 扩展执行，人保留判断：** 解决如何让 Agent 多做事，但目标、标准、验收和最终责任仍由人掌握的问题。
- **依附关系——结果在推进，判断者却缺席：** 解决人无法解释、验证和接管任务，却仍然持续接受 AI 输出的问题。
- **导学关系——不仅完成任务，还要形成能力：** 解决代码能够运行，但工程师没有真正理解和迁移能力的问题。

- **内观关系——借助 AI 校准自己的判断：** 解决工程师看不见自身假设、偏好和决策盲区的问题。
- **四种关系的切换：** 解决把四种关系理解为固定人群或能力等级，不能根据任务主动调整的问题。

---

3. 面向未来一年：AI Agent 会如何发展，AI Agent 工程师将面临什么挑战？

> 本课目标：帮助学员看清未来一年 Agent 技术的主要方向，并将个人能力建立在长期稳定的工程问题上。

- **从对话助手走向长时执行者：** 解决仍然用聊天机器人思维设计 Agent，无法支撑跨系统、跨时段复杂任务的问题。
- **从生成内容走向操作真实世界：** 解决 Agent 获得数据、工具和业务权限后，安全、副作用和审计风险快速增加的问题。
- **从单 Agent 走向多 Agent 与后台服务：** 解决多个 Agent 如何协作、交接状态、持续运行和主动推进任务的问题。
- **模型能力与 Harness 持续重构：** 解决把当前模型短板固化为复杂流程，导致系统不能随模型升级而演进的问题。
- **EvalOps 与行动安全成为基础设施：** 解决概率性系统无法仅靠上线前测试和内容审核保证生产质量的问题。
- **Agent 工程师的能力升级：** 解决只会调用模型和使用框架的问题，转向业务建模、Runtime、治理、安全、评测和系统责任。

4. DeepSeek Harness 架构拆解：新一代 Agent Harness 正在走向哪里？

> 本课目标：通过深度拆解 DeepSeek Harness 的架构设计，并与 Claude Code 和课程 DataAgent 企业级架构进行对照，理解新一代 Harness 的核心设计取舍、创新价值与适用边界，建立判断未来 Agent Harness 技术演进的架构视角。

- **从 Agent Framework 到 Agent Harness**：为什么 DeepSeek 要重新设计一套 Harness？
- **深入 DSH 核心架构**：它做了哪些不同于传统 Agent Framework 的设计选择？
- **DSH vs Claude Code**：两种 Harness 背后的产品目标与架构哲学有什么不同？
- **DSH vs DataAgent**：通用 Harness 与企业级业务 Agent Runtime 的边界在哪里？
- **从 DSH 看 Harness 的架构创新**：哪些设计值得企业级 Agent 借鉴？
- **从 DSH 看未来 Harness 生态**：Agent 基础设施可能发生什么变化？
- **课程最终架构讨论**：如果今天重新设计 DataAgent，我们会改变什么？